

Descrierea Software.

Subrutinele în limbaj de asamblare

1.1. Subrutinele de programare ale circuitelor 8251 și 8255

1.1.1. Subrutina de programarea a interfeței seriale

serial_prog:

```

MOV DX, 0AF0H      ; Adresa portului de comenzi pentru serială
MOV AL, 40H        ; Resetare USART
OUT DX, AL         ; Trimite comanda de resetare
MOV AL, 11001100B  ; 8 biți, fără paritate, 1 stop bit, asincron
OUT DX, AL         ; Configurează modul de lucru
MOV AL, 00010101B  ; Cuvânt de comandă
OUT DX, AL         ; Trimite comanda de activare

```

RET

1.1.2. Subrutina de programarea a interfeței paralele

paralel_prog:

```

MOV DX, 0D76       ; Adresa portului de comandă RCC
MOV AL, 10000000B  ; Configurare: Port A - Output, Port B -
                    ; Input, Port C - Split
OUT DX, AL         ; Trimite configurația RCC

```

RET

1.2. Subrutinele de emisie / recepție caracter pe interfața serială

1.2.1. Subrutina de transmisie a unui caracter pe interfața serială

serial_transm:

```

MOV DX, 04d2h      ; Portul de date serială (variantă 1)

```

W_T_S:

```

    IN AL, DX          ; Citire status din port
    TEST AL, 20h       ; Verificăm dacă bufferul de transmisie e gol
                        ; (bit 5)
    JZ W_T_S           ; Așteaptă până când bitul 5 devine 1

                        ; Portul de date
    MOV DX, 04d2h
    MOV AL, [CHAR]     ; Caracterul de transmis din memorie
    OUT DX, AL         ; Transmite caracterul

```

RET

1.2.2. Subrutina de recepție a unui caracter pe interfața serială

serial_rec:

```

    MOV DX, 0AF2h      ; Portul de date serială (variantă 1)

```

W_R_S:

```

    IN AL, DX          ; Citire status din port
    TEST AL, 10        ; Verificăm dacă datele sunt disponibile (bit
                        ; 4)
    JZ W_R_S           ; Așteaptă până când bitul 4 devine 1

                        ; Portul de date
    MOV DX, 0AF2h
    IN AL, DX          ; Citește caracterul recepționat
    MOV [CHAR], AL     ; Stochează caracterul în memorie

```

RET

1.3. Subrutina de emisie a unui caracter pe interfața paralelă

paralel_emisie:

```

    MOV DX, 0250h      ; Portul A (variantă 1)
    MOV AL, CL         ; Preluăm caracterul din registrul CL
    OUT DX, AL         ; Transmitem caracterul pe port

```

RET

1.4. Subrutina de scanare a minitastaturii

tastatura_scan:

```

col_1:                                ; Punem 0 pe prima coloană și se verifică dacă
                                        s-au acționat tastele 1, 4, sau 7

    MOV AL, 0840H                      ; se pune 0 pe prima coloana (0111 1111 B)
    OUT 0100H, AL                      ; selectez ST1 (ieșirea tastaturii)

    IN AL, 08C0H                       ; încarc conținutul aflat la portul /ST2
    AND AL, 80H                        ; verific prima poziție
    JZ TASTA1                          ; salt la tasta 1 dacă rezultatul este 0

    IN AL, 08C0H                       ; verific a doua poziție
    AND AL, 40H                        ; salt la tasta 4 dacă rezultatul este 0
    JZ TASTA4

    IN AL, 08C0H                       ; verific a treia poziție
    AND AL, 20H                        ; salt la tasta 7 dacă rezultatul este 0
    JZ TASTA7

col_2:                                ; Punem 0 pe a doua coloană și se verifică
                                        dacă s-au acționat tastele 2, 5, sau 8

    MOV AL, 0840H                      ; se pune 0 pe prima coloana (0111 1111 B)
    OUT 0100H, AL                      ; selectez ST1 (ieșirea tastaturii)

    IN AL, 08C0H                       ; încarc conținutul aflat la portul /ST2
    AND AL, 80H                        ; verific prima poziție
    JZ TASTA2                          ; salt la tasta 2 dacă rezultatul este 0

```

JZ TASTA2

```
IN AL, 08C0H      ; verific a doua pozitie
AND AL, 40H        ; salt la tasta 5 daca rezultatul este 0
JZ TASTA5
```

```
IN AL, 08C0H      ; verific a treia pozitie
AND AL, 20H        ; salt la tasta 8 daca rezultatul este 0
JZ TASTA8
```

```
col_3:            ; Punem 0 pe a treia coloana si se verifica
                  ; daca s-au actionat tastele 3, 6, sau 9
```

```
                  ; se pune 0 pe prima coloana (0111 1111 B)
MOV AL, 0840H      ; selectez ST1 (iesirea tastaturii)
```

```
OUT 0100H, AL      ; incarc continutul aflat la portul /ST2
```

```
IN AL, 08C0H      ; verific prima pozitie
AND AL, 80H        ; salt la tasta 3 daca rezultatul este 0
JZ TASTA3
```

```
IN AL, 08C0H      ; verific a doua pozitie
AND AL, 40H        ; salt la tasta 6 daca rezultatul este 0
JZ TASTA6
```

```
IN AL, 08C0H      ; verific a treia pozitie
AND AL, 20H        ; salt la tasta 9 daca rezultatul este 0
JZ TASTA9
```

1.5. Subrutinele de aprindere / stingere a unui LED

1.5.1. Subrutina de aprindere a unui LED

LED_aprinde:

```
MOV DX, 0C40H

MOV AL, FFEH      ; ia valoarea în funcție de led-ul care se
                  ; vrea aprins

OUT DX, AL
```

RET

1.5.2. Subrutina de stingere a unui LED

LED_stinge:

```
MOV AL, FFFH      ; stingem totul (punem pe „1”)

OUT DX, AL
```

RET

1.6. Rutina de afișare a unui caracter hexa pe un rang cu segmente

```
AFISARE_HEX:      ; CL conține caracterul hexazecimal de afișat
                  ; CH conține rangul afișajului (al câtelea
                  ; afișaj va fi folosit)

MOV BL, CL
SUB BL, 30H       ; Copiem numărul hexazecimal în BL
```

```
MAPARE_HEX:
CMP BL, 0         ; Verificăm dacă este cifra 0
JE TABEL_0
CMP BL, 1         ; Verificăm dacă este cifra 1
JE TABEL_1
CMP BL, 2         ; Verificăm dacă este cifra 2
JE TABEL_2
                  ; Verificăm dacă este cifra 3
```

```

CMP BL, 3
JE TABEL_3          ; Verificăm dacă este cifra 4
CMP BL, 4
JE TABEL_4          ; Verificăm dacă este cifra 5
CMP BL, 5
JE TABEL_5          ; Verificăm dacă este cifra 6
CMP BL, 6
JE TABEL_6          ; Verificăm dacă este cifra 7
CMP BL, 7
JE TABEL_7          ; Verificăm dacă este cifra 8
CMP BL, 8
JE TABEL_8          ; Verificăm dacă este cifra 9
CMP BL, 9
JE TABEL_9          ; Verificăm dacă este litera A
CMP BL, 'A'
JE TABEL_A          ; Verificăm dacă este litera B
CMP BL, 'B'
JE TABEL_B          ; Verificăm dacă este litera C
CMP BL, 'C'
JE TABEL_C          ; Verificăm dacă este litera D
CMP BL, 'D'
JE TABEL_D          ; Verificăm dacă este litera E
CMP BL, 'E'
JE TABEL_E          ; Verificăm dacă este litera F
CMP BL, 'F'
JE TABEL_F          ; 0: a, b, c, d, e, f aprinse
TABEL_0:
MOV AL, 0x3F

```

```
RET ; 1: b, c aprinse
TABEL_1:
MOV AL, 0x06
RET ; 2: a, b, d, e, g aprinse
TABEL_2:
MOV AL, 0xB6
RET ; 3: a, b, c, d, g aprinse
TABEL_3:
MOV AL, 0xB3
RET ; 4: b, c, f, g aprinse
TABEL_4:
MOV AL, 0x66
RET ; 5: a, c, d, f, g aprinse
TABEL_5:
MOV AL, 0xB5
RET ; 6: a, c, d, e, f, g aprinse
TABEL_6:
MOV AL, 0xF5
RET ; 7: a, b, c aprinse
TABEL_7:
MOV AL, 0x07
RET ; 8: a, b, c, d, e, f, g aprinse
TABEL_8:
MOV AL, 0x7F
RET ; 9: a, b, c, d, f, g aprinse
TABEL_9:
MOV AL, 0xB7
RET ; A: a, b, c, e, f, g aprinse
```

```
TABEL_A:
    MOV AL, 0x77
    RET                ; B: c, d, e, f, g aprinse

TABEL_B:
    MOV AL, 0xF6
    RET                ; C: a, d, e, f aprinse

TABEL_C:
    MOV AL, 0xD1
    RET                ; D: b, c, d, e, g aprinse

TABEL_D:
    MOV AL, 0xF3
    RET                ; E: a, d, e, f, g aprinse

TABEL_E:
    MOV AL, 0xD5
    RET                ; F: a, e, f, g aprinse

TABEL_F:
    MOV AL, 0x75        ; Selectăm afișajul corespunzător rangului
                        ; (CH)
                        ; Portul de date al afișajului cu 7 segmente
    MOV DX, 0AF0h        ; Adăugăm rangul pentru a selecta portul
                        ; correct
    ADD DX, CH           ; Trimitem codul de segmente la portul de date

    OUT DX, AL
    RET
```